

Fast Realistic Rendering

Physically Based Rendering

PROJECT STATEMENT

In this assignment, we will deal with advanced rendering techniques. The goal is to get familiar with the elements that are under the hood of Physically Based Rendering in real-time. To do so, you have to implement a method based on the microfacet approximation to non-optically flat surfaces using Image Based Lighting techniques.

Particularly, we ask you for:

1. [10%] Implementing a basic visualization of the object. This basic visualization must allow the user to change how the surface is rendered between:
 - a. Material color/texture (include your own) using Phong Shading
 - b. Roughness map (include your own)
 - c. Metalness map (include your own)
2. [15%] Implementing an environment reflection shader using a cube texture.
3. [30%] Implementing a PBR model using a point light that changes its behavior depending on the material's roughness and metalness. It should work using:
 - a. global values
 - b. metalness and roughness maps.
4. [25%] Implementing an image-based lighting approach based on the use of cube maps that contain, on one hand, the diffuse irradiance map and, on the other hand, the cosine weighted specular convolutions. Remember that material roughness (described on the roughness map) must be a parameter for the pre-computation and that metalness (described on the metalness map) must be a parameter for the computation.

IMPORTANT! Both solutions must:

- a. Approximate the implicit geometry function using the microfacet theory.
 - b. Employ the Schlick's approximation to Fresnel reflectance.
 - c. Study the differences between linear computation and gamma correction.
5. [20%] You will need to implement some application/code to precompute the filtered cube maps for (4.b). Before you have this code working, you can find Cube Maps to use here: <http://www.humus.name/index.php?page=Textures>

Note: Percentages roughly describe the load of work and the importance of each part. The grade may not correspond to them.

DELIVERY

Please upload a single zip file by **April 28th**, named using “PBR” and your username. For instance: PBR-imanol.munoz.zip. The zip file should contain:

- A compilable and executable project. This includes:
 - All the required .c, .cc, .cpp, .h, .hpp, .ui, etc. files needed to compile your application (including the map generator).
 - All the code used by your shaders.
 - All the used texture files.
 - A .PRO file, Makefile, CMakeLists, or similar script that can compile and generate an executable file out of your source files.
 - A list of the dependencies needed to compile your application.
- A short report explaining the implemented solution, delivered in PDF format following the tex template. The report may include:
 - A general explanation of techniques used. You can use draws to illustrate them.
 - A very well-documented version of your shaders. It should be clear which lines of the code relate to the different parts of the algorithms.
 - A discussion of your project where you describe the problems found and the proposed solutions. You can use images/screenshots to illustrate them.
- A video presentation of your project (it can be a link to a cloud service like Youtube). This video presentation will include a demonstration execution of the functionalities of your code described by you with a voice-over. You need to discuss results, limitations, and anything you find relevant for your grade. Google meet can help you with that.

Remember:

You can find several examples of how to implement this kind of algorithm on the Internet. **Any code directly copied from or delivering a mimetic solution (without a fair justification) to existing examples will imply failing the assignment.**

RESOURCES

To get familiar with them, you can check the following resources:

1. Siggraph Physically Based Shading in Theory and Practice course slides:
http://blog.selfshadow.com/publications/s2014-shading-course/hoffman/s2014_pbs_physics_math_slides.pdf
2. Siggraph Physically Based Shading in Theory and Practice course notes:
http://blog.selfshadow.com/publications/s2013-shading-course/hoffman/s2013_pbs_physics_math_notes.pdf
3. GPU Gems on Image-Based Lighting:
https://developer.nvidia.com/gpugems/GPUGems/gpugems_ch19.html